```
--------------------------------------------------------------------------------
--
--  Switch-activated PWM Sound in VHDL
--
--  This is an implementation of the EE119a PWM board functionality in VHDL.
--  There are four switches on the board, each of which triggers a distinct
--  message ranging in length from 2 seconds to 18 seconds, sampled at 8 KHz.
--  The messages are stored on a 512Kx8 EPROM.
--  If a switch is still pressed when the message completes, then the system
--  plays the message again. Switches are prioritized in descending order when
--  two are pressed at the same time. Pressing a switch when a message is
--  already playing will not trigger the new sound to play.
--  This code fits onto the Xilinx version.
--  It uses a behavioral type architecture.
--
--  Entities included are:
--     PWMSystem      - plays PWM sound when a switch on the board is activated
--
--  Input:             CLK          system clock
--                     SW1 to SW4   switch inputs
--                     AudioData    8 bit input audio sample data
--  Output:            AudioAddr    12 bit output EPROM address
--                     AudioPWMOut  1 bit output to PWM speaker
--
--  Revision History:
--     06 Nov 26  Emily Wu          Initial Revision
--     07 Nov 26  Emily Wu          Merged into 1 entity; changed inputs/outputs
--     09 Nov 26  Emily Wu          - Added cases for repeating the same message
--                                  - Switched from 8, 4, & 20 bit ctrs to one
--                                      12 bit counter
--     11 Nov 26  Emily Wu          Added comments
--------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- ###########################################################################
-- PWMSystem entity definition

entity PWMSystem is port(
    clk             :   in  std_logic;
    SW1             :   in  std_logic;
    SW2             :   in  std_logic;
    SW3             :   in  std_logic;
    SW4             :   in  std_logic;
    AudioData       :   in  std_logic_vector(7 downto 0);
    AudioAddr       :   out std_logic_vector(18 downto 0);
    AudioPWMOut     :   out std_logic
    );
end PWMSystem;

--
-- PWMSystem behavioral architecture
--
```

```
architecture PWMSystemArchitecture of PWMSystem is
    -- CONSTANTS --
    -- EPROM starting addresses
    constant startAddrSW1 : unsigned(18 downto 0) := to_unsigned(16#40000#, 19);
    constant startAddrSW2 : unsigned(18 downto 0) := to_unsigned(16#48000#, 19);
    constant startAddrSW3 : unsigned(18 downto 0) := to_unsigned(16#58000#, 19);
    constant startAddrSW4 : unsigned(18 downto 0) := to_unsigned(16#7C000#, 19);
    -- EPROM ending addresses
    constant endAddrSW1   : unsigned(18 downto 0) := to_unsigned(16#47FFF#, 19);
    constant endAddrSW2   : unsigned(18 downto 0) := to_unsigned(16#57FFF#, 19);
    constant endAddrSW3   : unsigned(18 downto 0) := to_unsigned(16#7BFFF#, 19);
    constant endAddrSW4   : unsigned(18 downto 0) := to_unsigned(16#7FFFF#, 19);
    -- each sample cycles from 0 to 4095 (4096 CPLD clocks)
    constant sampleMax    : unsigned(11 downto 0) := to_unsigned(4095, 12);

    -- SIGNALS --
    signal currAddr       : unsigned(18 downto 0) := (others => '0');
    signal playing        : std_logic := '0';
    signal sample         :  unsigned(7 downto 0);

    signal activeSwitch   : integer range 1 to 4 := 4;
    signal switchStates   : std_logic_vector(4 downto 1) := (others => '0');
    signal messageEnded   : std_logic;
    signal startAddr      : unsigned(18 downto 0);

    signal enSwitchCheck  : std_logic := '0';
    signal count12bit     : unsigned(11 downto 0);

begin

-- ######################## Switches ###############################################
-- Switch-state related processes
-- Special Behavior: when two switches are pressed simultaneously, the following
--      priority list is followed: SW4 > SW3 > SW2 > SW1

    getSwitchStates: process(clk)
    begin
        if rising_edge(clk) then
            switchStates(4) <= SW4;
            switchStates(3) <= SW3;
            switchStates(2) <= SW2;
            switchStates(1) <= SW1;
        end if;
    end process getSwitchStates;

    -- set the starting address for this switch
    getStartAddr: process(switchStates)
    begin
        if switchStates(4) = '1' then
            startAddr <= startAddrSW4;
        elsif switchStates(3) = '1' then
            startAddr <= startAddrSW3;
        elsif switchStates(2) = '1' then
            startAddr <= startAddrSW2;
```

```
        elsif switchStates(1) = '1' then
            startAddr <= startAddrSW1;
        end if;
    end process getStartAddr;

    -- check if the currently playing message has ended
    -- this is true if any of the ending address has been reached.
    messageEnd: process(clk)
    begin
        if rising_edge(clk) then
            if (currAddr = endAddrSW1) or
                (currAddr = endAddrSW2)  or
                (currAddr = endAddrSW3) or
                (currAddr = endAddrSW4) then
                    messageEnded <= '1';
            else messageEnded <= '0';
            end if;
        end if;
    end process messageEnd;

-- ###################### Counter ##############################################
-- 12 bit synchronous counter
-- Increments on every rising clock edge, except when a cycle sample has ended,
--      in which case we should set EnSwitchCheck high (check if any switch has
--      been pressed)
 countSample: process(clk)
    begin
        if rising_edge(clk) then
            count12bit <= count12bit + 1;
            if count12bit = sampleMax - 1 then
                enSwitchCheck <= '1';
            else enSwitchCheck <= '0';
            end if;
        end if;
    end process countSample;

-- ###################### Main Behavior ########################################
-- 1. Appropriately sets the current address when a switch is pressed
-- 2. Sets the active switch indicator signal (holds which switch was pressed)
-- 3. At the end of a message, if the same switch is still pressed, then reset
--      the current address to the same original start address (otherwise stop
--      playing)
-- 4. OTHERWISE, the only option left is that a message is currently playing
--      so we store the retrieved AudioData sample and implement pipelining by
--      incrementing to the next EPROM address ahead of time.

    main: process(clk)
    begin
        if rising_edge(clk) then
            if enSwitchCheck = '1' then
                if playing = '0' then
                    if switchStates /= "0000" then
                        currAddr <= startAddr;
                        if switchStates(4) = '1' then
                                activeSwitch <= 4;
```

```
                        elsif switchStates(3) = '1' then
                            activeSwitch <= 3;
                        elsif switchStates(2) = '1' then
                            activeSwitch <= 2;
                        else
                            activeSwitch <= 1;
                        end if;
                    playing <= '1';
                end if;

            elsif messageEnded = '1' then
                if switchStates(activeSwitch) = '1' then
                    currAddr <= startAddr;
                else playing <= '0';
                end if;

            else
                sample <= unsigned(AudioData);
                currAddr <= currAddr + 1;
            end if;
        end if;
    end if;
end process main;

-- ###################### Comparator ##########################################
-- Compares the 12 bit counter value (bottom 8 bits only) to the data read from
-- the EPROM
-- Special Behavior: Since the assignment did not specify what would happen when
-- the sample is equal to the counter, we have selected to set counter = 1 when
-- they are equal.
-- The comparator additionally sets the output to 0 when nothing is playing.
    Comparator: process(clk)
    begin
        if rising_edge(clk) then
            if playing = '0' then
                AudioPWMOut <= '0';
            elsif count12bit(7 downto 0) > sample then
                AudioPWMOut <= '0';
            elsif (count12bit(7 downto 0) < sample) or
                  (count12bit(7 downto 0) = sample) then
                AudioPWMOut <= '1';
            else
                null;
            end if;
        end if;
    end process Comparator;

    AudioAddr  <= std_logic_vector(currAddr);


end PWMSystemArchitecture;
```